EXPERIMENT NO: 6

PROGRAM TO IMPLEMENT ERROR DETECTION AND CORRECTION USING HAMMING CODE

AIM

To write a program to implement error detection and correction using the Hamming code.

ALGORITHM

1. Read the input text from input.txt.
2. Convert each character to binary and add parity bits at positions 1, 2, 4 and 8.
3. Store the encoded data in channel.txt.
4. At the receiver, read the data and calculate the syndrome to find the error position.
5. Correct the bit if an error is found.
6. Remove the parity bits and display the original text.

PROGRAM - SENDER (sender.c)

```
#include <stdio.h>

/* Hamming(12,8): parity bits at positions 1,2,4,8 - data bits fill the rest, MSB
first */
static int is_parity_pos(int p) { return p == 1 || p == 2 || p == 4 || p == 8; }

void encode_byte(unsigned char byte, FILE *out)
{
    int code[13] = {0};
    int k = 7;                                  /* start from MSB of the byte */

    for (int pos = 1; pos <= 12; pos++)
        if (!is_parity_pos(pos))
            code[pos] = (byte >> k--) & 1;

    /* each parity bit covers every position whose index has that bit set */
    for (int p = 1; p <= 8; p <<= 1) {
        int parity = 0;
        for (int pos = 1; pos <= 12; pos++)
            if ((pos & p) && pos != p)
                parity ^= code[pos];
        code[p] = parity;
    }

    for (int pos = 1; pos <= 12; pos++)
        fputc('0' + code[pos], out);
    fputc('\n', out);
}

int main(void)
{
    FILE *in  = fopen("input.txt", "r");
    FILE *out = fopen("channel.txt", "w");
    int c, count = 0;

    if (!in || !out) {
```

```
        printf("Could not open input.txt / channel.txt\n");
        return 1;
    }
    while ((c = fgetc(in)) != EOF && c != '\n') {
        encode_byte((unsigned char)c, out);
        count++;
    }
    fclose(in);
    fclose(out);
    printf("%d characters encoded into %d-bit codewords -> channel.txt\n", count,
12);
    return 0;
}
```

PROGRAM - RECEIVER (receiver.c)

```
#include <stdio.h>

static int is_parity_pos(int p) { return p == 1 || p == 2 || p == 4 || p == 8; }

/* returns decoded byte; *syn receives the syndrome (0 = no error) */
int decode_word(const char *w, int *syn)
{
    int code[13] = {0}, s = 0, byte = 0, k = 7;

    for (int pos = 1; pos <= 12; pos++)
        code[pos] = w[pos - 1] - '0';

    /* syndrome = XOR of the positions of all bits that are 1 */
    for (int pos = 1; pos <= 12; pos++)
        if (code[pos])
            s ^= pos;
    *syn = s;

    if (s >= 1 && s <= 12)
        code[s] ^= 1;                           /* flip the faulty bit */

    for (int pos = 1; pos <= 12; pos++)
        if (!is_parity_pos(pos))
            byte |= code[pos] << k--;
    return byte;
}

int main(void)
{
    FILE *ch = fopen("channel.txt", "r");
    char word[32];
    int syn, blk = 0;

    if (!ch) { printf("channel.txt not found\n"); return 1; }

    printf("Received text : ");
    char log[2048] = ""; int off = 0;
    while (fscanf(ch, "%31s", word) == 1) {
        blk++;
        char c = (char)decode_word(word, &syn);
        putchar(c);
        if (syn)
```

```
            off += snprintf(log + off, sizeof log - off,
                            "  -> Block %d: syndrome = %d, bit %d corrected\n",
blk, syn, syn);
    }
    printf("\n");
    if (off) printf("Error report:\n%s", log);
    else     printf("No errors detected.\n");
    fclose(ch);
    return 0;
}
```

INPUT

input.txt contains: NETWORK

OUTPUT

```
7 characters encoded into 12-bit codewords -> channel.txt
Received text : NETWORK
No errors detected.
```

With an error introduced in the channel file:

```
Received text : NETWORK
Error report:
  -> Block 3: syndrome = 7, bit 7 corrected
  -> Block 6: syndrome = 11, bit 11 corrected
```

RESULT

The Hamming code error detection and correction program was successfully implemented and tested.